

PyTorch基础

视听信息系统导论



Pytorch的基础使用

- 基本配置
- 数据读入
- 模型构建
- 模型初始化
- 损失函数
- 训练与评估
- 优化器
- 可视化



1. PyTorch基本配置



常见的需要导入的包

```
import os
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import torch.optim as optimizer
```

注意里只是建议导入的包导入的方式，可以采用不同的方案，比如涉及到表格信息的读入很可能用到pandas，对于不同的项目可能还需要导入一些更上层的包如cv2等。如果涉及可视化还会用到matplotlib、seaborn等。涉及到下游分析和指标计算也常用到sklearn

常见的超参数

```
batch_size = 16  
# 批次的大小  
lr = 1e-4  
# 优化器的学习率  
max_epochs = 100
```

- batch size: 读取数据时每一个批次的大小
- lr: 优化器的学习率
- max_epochs: 训练的轮数

指定设备

```
# 方案一：使用os.environ, 这种情况如果使用GPU不需要设置
import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0,1' # 指明调用的GPU为0,1号

# 方案二：使用“device”, 后续对要使用GPU的变量用.to(device)即可
device = torch.device("cuda:1" if torch.cuda.is_available() else "cpu") # 指明调用的GPU为1
```

- 我们的数据和模型如果没有经过显式指明设备，默认会存储在CPU上，为了加速模型的训练，我们需要显式调用GPU，一般情况下GPU的设置有以下两种常见的方式，分别是通过环境变量来指定和通过设置device，后续对需要使用GPU的变量使用.to(device)

2. 数据读入



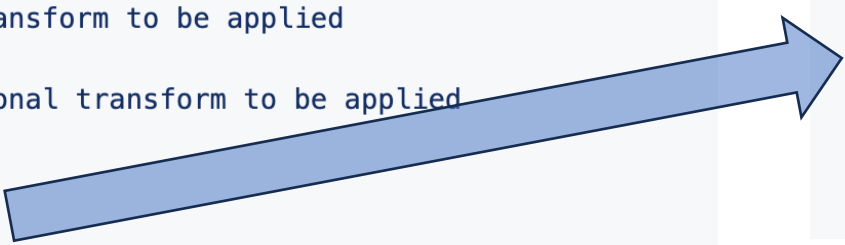
构建自己的Dataset

- 我们可以定义自己的Dataset类来实现灵活的数据读取，定义的需要继承PyTorch自身的Dataset类。主要包含三个函数：
 - `__init__`: 用于向类中传入外部参数，同时定义样本集

```
import os
import pandas as pd
from torchvision.io import read_image

class MyDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        """
        Args:
            annotations_file (string): Path to the csv file with annotations.
            img_dir (string): Directory with all the images.
            transform (callable, optional): Optional transform to be applied
                on a sample.
            target_transform (callable, optional): Optional transform to be applied
                on the target.
        """
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform
```

其中标签文件形式为:



```
image1.jpg, 0
image2.jpg, 1
.....
image9.jpg, 9
```


构建自己的Dataset

- 我们可以定义自己的Dataset类来实现灵活的数据读取，定义的类需要继承PyTorch自身的Dataset类。主要包含三个函数：
 - `__getitem__`: 用于逐个读取样本集合中的元素，可以进行一定的变换，并将返回训练/验证所需的数据

```
def __getitem__(self, idx):  
    """  
    Args:  
        idx (int): Index  
    """  
    img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])  
    image = read_image(img_path)  
    label = self.img_labels.iloc[idx, 1]  
    if self.transform:  
        image = self.transform(image)  
    if self.target_transform:  
        label = self.target_transform(label)  
    return image, label
```

构建自己的Dataset

- 我们可以定义自己的Dataset类来实现灵活的数据读取，定义类需要继承PyTorch自身的Dataset类。主要包含三个函数：
 - `__len__`: 用于返回数据集的样本数

```
def __len__(self):  
    return len(self.img_labels)
```

读取数据

- 在训练过程中，通常使用DataLoader来按照批次读取数据

```
from torch.utils.data import DataLoader

train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size, num_workers=4, shuffle=True, drop_last=True)
val_loader = torch.utils.data.DataLoader(val_data, batch_size=batch_size, num_workers=4, shuffle=False)
```

- Batch_size: 样本是按批读入的，可以一次多很多个数据，batch_size就是每次读入的样本数
- num_workers: 有多少个进程用于读取数据，根据自己的电脑配置来设置
- shuffle: 是否将读入的数据打乱，一般在训练集中设置为True，验证集和测试集中设置为False
- drop_last: 对于样本最后一部分没有达到批次数量的样本，使其不再参与训练，该设置一般仅用于训练

3. 模型构建



神经网络的构造

- 所有神经网络的模块都继承自`torch.nn.Module`，我们可以继承它来定义我们想要的模型。在自定义模型时，一般需要重载`__init__`函数和`forward`函数，定义好后，系统通过自动求梯度而自动生成反向传播所需的`backward`函数。

```
import torch
from torch import nn

class MLP(nn.Module):
    # 声明带有模型参数的层，这里声明了两个全连接层
    def __init__(self, **kwargs):
        # 调用MLP父类Block的构造函数来进行必要的初始化。这样在构造实例时还可以指定其他函数
        super(MLP, self).__init__(**kwargs)
        self.hidden = nn.Linear(784, 256)
        self.act = nn.ReLU()
        self.output = nn.Linear(256, 10)

    # 定义模型的前向计算，即如何根据输入x计算返回所需要的模型输出
    def forward(self, x):
        o = self.act(self.hidden(x))
        return self.output(o)
```

神经网络的实例化

- 我们可以实例化MLP类别得到模型变量net，以下代码为实例化MLP类，并传入输入数据X做一次前向计算。其中net(X)会调用MLP类中的__call__函数，__call__函数将继续调用MLP类别中的forward函数，完成前向计算。

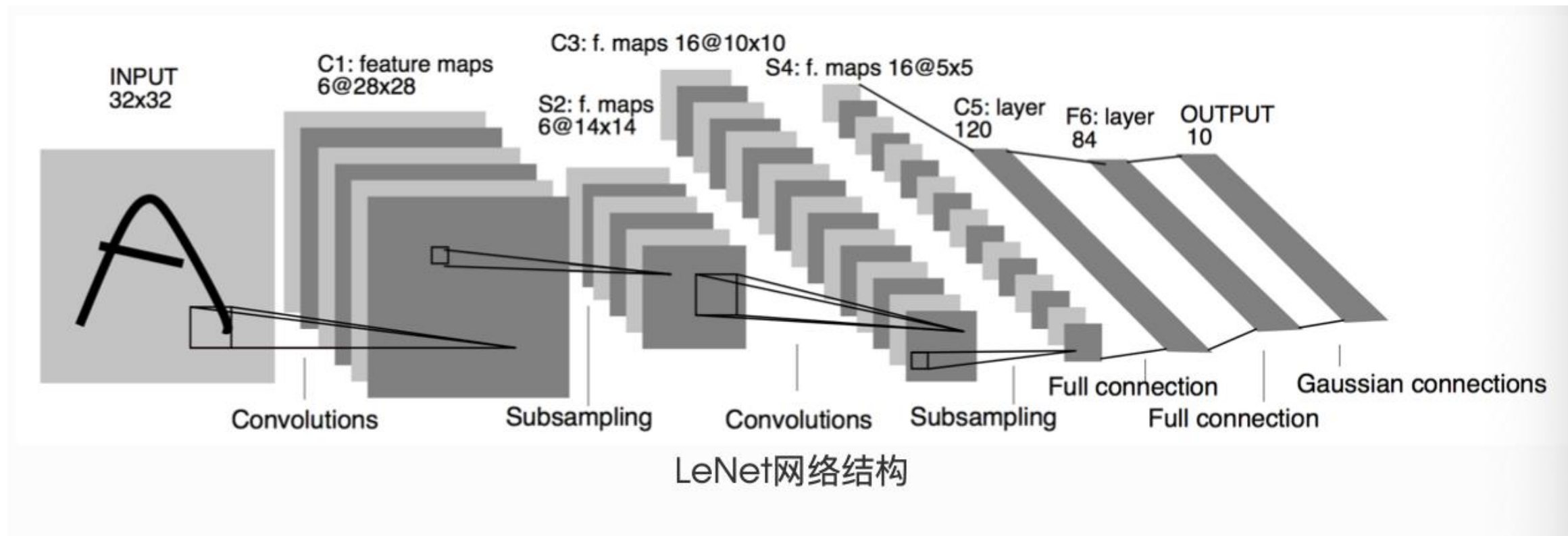
```
X = torch.rand(2,784) # 设置一个随机的输入张量
net = MLP() # 实例化模型
print(net) # 打印模型
net(X) # 前向计算
```

```
MLP(
  (hidden): Linear(in_features=784, out_features=256, bias=True)
  (act): ReLU()
  (output): Linear(in_features=256, out_features=10, bias=True)
)
tensor([[ 0.0149, -0.2641, -0.0040,  0.0945, -0.1277, -0.0092,  0.0343,  0.0627,
          -0.1742,  0.1866],
        [ 0.0738, -0.1409,  0.0790,  0.0597, -0.1572,  0.0479, -0.0519,  0.0211,
          -0.1435,  0.1958]], grad_fn=<AddmmBackward>)
```

神经网络中的常见层

- 带有可学习参数的层：
 - `nn.Linear` : 全连接层, 包含可学习的权重和偏置
 - `nn.Conv2d` : 卷积层, 用于处理二维输入数据, 包含可学习的卷积核和偏置
 - `nn.BatchNorm2d` : 二维批标准化层, 用于在卷积或全连接层之后进行批标准化, 包含可学习的缩放因子和偏置
 - `nn.LayerNorm` : 层标准化
- 不带有可学习参数的层：
 - `nn.ReLU` : ReLU激活函数层
 - `nn.Sigmoid` : sigmoid激活函数层
 - `nn.Tanh` : tanh激活函数层
 - `nn.MaxPool2d` : 二维最大池化层
 - `nn.Dropout` : 随机失活层, 用于在训练过程中丢弃部分神经元防止过拟合
 - `nn.Flatten` : 扁平化层, 用于将多维输入数据展平成1维

模型示例



模型示例

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):

    def __init__(self):
        super(Net, self).__init__()
        # 输入图像channel: 1; 输出channel: 6; 5x5卷积核
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        # an affine operation: y = Wx + b
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        # 2x2 Max pooling
        x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
        # 如果是方阵, 则可以只使用一个数字进行定义
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = x.view(-1, self.num_flat_features(x))
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

    def num_flat_features(self, x):
        size = x.size()[1:] # 除去批处理维度的其他所有维度
        num_features = 1
        for s in size:
            num_features *= s
        return num_features
```

4. 模型初始化



初始化参数

- 在深度学习模型的训练中，权重的初始值极为重要。一个好的初始值，会使模型收敛速度提高，使模型准确率更精确。一般情况下，我们不使用全0初始值训练网络。为了利于训练和减少收敛时间，我们需要对模型进行合理的初始化。PyTorch也在`torch.nn.init`中为我们提供了常用的初始化方法。

torch.nn.init内容

- torch.nn.init中提供了多种初始化方法
 - torch.nn.init.uniform_
 - torch.nn.init.normal_
 - torch.nn.init.constant_
 - torch.nn.init.ones_
 - torch.nn.init.zeros_
 - torch.nn.init.eye_
 - torch.nn.init.xavier_uniform_
 - torch.nn.init.kaiming_normal_
- 更多细节见torch.nn.init官方文档: <https://pytorch.org/docs/stable/nn.init.html>

torch.nn.init的使用

- 通常根据实际模型来使用torch.nn.init进行初始化，判断模型用isinstance来进行模块判断

```
import torch
import torch.nn as nn

conv = nn.Conv2d(1,3,3)
linear = nn.Linear(10,1)

isinstance(conv,nn.Conv2d) # 判断conv是否是nn.Conv2d类型
isinstance(linear,nn.Conv2d) # 判断linear是否是nn.Conv2d类型
```

```
True
False
```

torch.nn.init的使用

- 对于不同类型的层，可以设置不同的权值初始化的方法

```
# 查看随机初始化的conv参数
```

```
conv.weight.data
```

```
# 查看linear的参数
```

```
linear.weight.data
```

```
tensor([[[[ 0.1174,  0.1071,  0.2977],
           [-0.2634, -0.0583, -0.2465],
           [ 0.1726, -0.0452, -0.2354]]],
        [[[ 0.1382,  0.1853, -0.1515],
           [ 0.0561,  0.2798, -0.2488],
           [-0.1288,  0.0031,  0.2826]]],
        [[[ 0.2655,  0.2566, -0.1276],
           [ 0.1905, -0.1308,  0.2933],
           [ 0.0557, -0.1880,  0.0669]]]])
```

```
tensor([[-0.0089,  0.1186,  0.1213, -0.2569,  0.1381,  0.3125,  0.1118, -0.0063, -0.2330,  0.1956]])
```

torch.nn.init的使用

- 对于不同类型的层，可以设置不同的权值初始化的方法

```
# 对conv进行kaiming初始化
torch.nn.init.kaiming_normal_(conv.weight.data)
conv.weight.data
# 对linear进行常数初始化
torch.nn.init.constant_(linear.weight.data,0.3)
linear.weight.data
```

```
tensor([[[[ 0.3249, -0.0500,  0.6703],
          [-0.3561,  0.0946,  0.4380],
          [-0.9426,  0.9116,  0.4374]]],
        [[ 0.6727,  0.9885,  0.1635],
          [ 0.7218, -1.2841, -0.2970],
          [-0.9128, -0.1134, -0.3846]]],
        [[ 0.2018,  0.4668, -0.0937],
          [-0.2701, -0.3073,  0.6686],
          [-0.3269, -0.0094,  0.3246]]]])
tensor([[0.3000, 0.3000, 0.3000, 0.3000, 0.3000, 0.3000, 0.3000, 0.3000, 0.3000,0.3000]])
```

初始化函数的封装

- 通常会将各种初始化方法定义成一个initialize_weights()的函数并在模型初始后进行使用

```
def initialize_weights(model):  
    for m in model.modules():  
        # 判断是否属于Conv2d  
        if isinstance(m, nn.Conv2d):  
            torch.nn.init.zeros_(m.weight.data)  
            # 判断是否有偏置  
            if m.bias is not None:  
                torch.nn.init.constant_(m.bias.data, 0.3)  
        elif isinstance(m, nn.Linear):  
            torch.nn.init.normal_(m.weight.data, 0.1)  
            if m.bias is not None:  
                torch.nn.init.zeros_(m.bias.data)  
        elif isinstance(m, nn.BatchNorm2d):  
            m.weight.data.fill_(1)  
            m.bias.data.zeros_()
```


初始化函数的使用

模型的定义

```
class MLP(nn.Module):
```

声明带有模型参数的层, 这里声明了两个全连接层

```
def __init__(self, **kwargs):
```

调用MLP父类Block的构造函数来进行必要的初始化。这样在构造实例时还可以指定其他函数

```
super(MLP, self).__init__(**kwargs)
```

```
self.hidden = nn.Conv2d(1,1,3)
```

```
self.act = nn.ReLU()
```

```
self.output = nn.Linear(10,1)
```

定义模型的前向计算, 即如何根据输入x计算返回所需要的模型输出

```
def forward(self, x):
```

```
o = self.act(self.hidden(x))
```

```
return self.output(o)
```

```
mlp = MLP()
```

```
print(mlp.hidden.weight.data)
```

```
print("-----初始化-----")
```

```
mlp.apply(initialize_weights)
```

```
# 或者initialize_weights(mlp)
```

```
print(mlp.hidden.weight.data)
```

```
tensor([[[[ 0.3069, -0.1865,  0.0182],  
          [ 0.2475,  0.3330,  0.1352],  
          [-0.0247, -0.0786,  0.1278]]]])
```

```
"-----初始化-----"
```

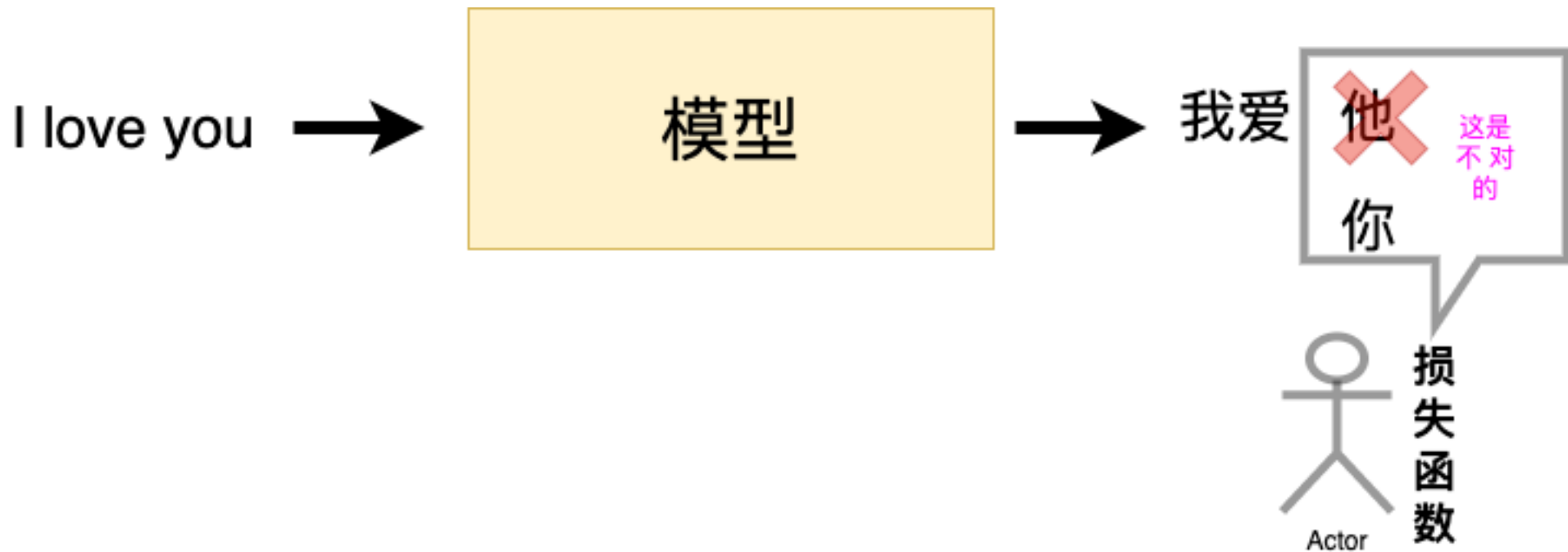
```
tensor([[[[0., 0., 0.],  
          [0., 0., 0.],  
          [0., 0., 0.]]]])
```

5. 损失函数



介绍

- 一个模型想要达到很好的效果需要学习，也就是我们常说的训练。一个好的训练离不开优质的负反馈，这里的损失函数就是模型的负反馈



L1 损失函数

```
torch.nn.L1Loss(size_average=None, reduce=None, reduction='mean')
```

- 计算输出 y 和真实标签 $target$ 之间的差值的绝对值
- $reduction$ 决定了计算模式, $none$ 是逐个计算, sum 为所有元素求和, $mean$ 为加权平均(默认)
- $L_n = |x_n - y_n|$

```
loss = nn.L1Loss()
input = torch.randn(3, 5, requires_grad=True)
target = torch.randn(3, 5)
output = loss(input, target)
output.backward()
print('L1损失函数的计算结果为', output)
```

L1损失函数的计算结果为 `tensor(1.5729, grad_fn=<L1LossBackward>)`

L1 损失函数

- 不同reduction计算结果示例

```
import torch
import torch.nn as nn

y = torch.Tensor([0,1,0,1,1])
target = torch.Tensor([1,0,0,1,0])
print(y)
print(target)
loss = nn.L1Loss(reduction='none')
print("none:", loss(y, target))
loss = nn.L1Loss(reduction='sum')
print("sum :", loss(y, target))
loss = nn.L1Loss(reduction='mean')
print("mean:", loss(y, target))
```

```
tensor([0., 1., 0., 1., 1.])
tensor([1., 0., 0., 1., 0.])
none: tensor([1., 1., 0., 0., 1.])
sum : tensor(3.)
mean: tensor(0.6000)
```

MSE 损失函数

```
torch.nn.MSELoss(size_average=None, reduce=None, reduction='mean')
```

- 计算输出 y 和真实标签 $target$ 之间的差值的平方
- $reduction$ 决定了计算模式, $none$ 是逐个计算, sum 为所有元素求和, $mean$ 为加权平均 (默认)
- $L_n = (x_n - y_n)^2$

```
loss = nn.MSELoss()  
input = torch.randn(3, 5, requires_grad=True)  
target = torch.randn(3, 5)  
output = loss(input, target)  
output.backward()  
print('MSE损失函数的计算结果为', output)
```

MSE损失函数的计算结果为 `tensor(1.6968, grad_fn=<MseLossBackward>)`

6. 优化器



介绍

- 深度学习的目标是通过不断改变网络参数，使得参数能够对输入做各种非线性变换拟合输出，本质上就是一个函数去寻找最优解，只不过这个最优解是一个矩阵，而如何快速求得这个最优解是深度学习研究的一个重点，以经典的resnet-50为例，它大约有2000万个系数需要进行计算，那么我们如何计算出这么多系数，有以下两种方法：
 - 第一种是直接暴力穷举一遍参数，这种方法从理论上行得通，但是实施上可能性基本为0，因为参数量过于庞大。
 - 为了使求解参数过程更快，人们提出了第二种办法，即BP（反向传播）+优化器逼近求解。

torch.optim

- torch.optim.SGD
- torch.optim.Adam
- torch.optim.AdamW
- 用法
 - `Optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)`
 - `Optimizer = torch.optim.Adam(model.parameters(), lr=0.1, betas=(0.9, 0.999))`
 - `Optimizer = torch.optim.Adam(model.parameters(), lr=0.1, betas=(0.9, 0.999), weight_decay=0.01)`

```
optimizer = torch.optim.SGD(net.parameters(), lr=1e-5)
for epoch in range(EPOCH):
    ...
    optimizer.zero_grad() #梯度置零
    loss = ...           #计算loss
    loss.backward()      #BP反向传播
    optimizer.step()     #梯度更新
```

7. 训练和评估



模型状态设置

```
model.train()    # 训练状态  
model.eval()     # 验证/测试状态
```

- `model.train()` 模式下模型中的一些特定层会采用不同的行为。如Dropout层在训练模式下，Dropout层会随机丢弃一部分神经元，以减少过拟合。BatchNormalization层在训练模式下会根据当前批次的统计信息对输入进行标准化，它计算批次的均值和方差，并将输入标准化为零均值和单位方差。这有助于加速训练过程并增强模型的泛化能力
- `model.eval()` 模式下，Dropout层不会进行随机丢弃，而是保留所有神经元的输出。BatchNormalization层使用在训练过程中学到的移动平均值和方差来标准化输入。

训练流程

- 从DataLoader中读取数据

```
for data, label in train_loader:
```

- 如有gpu, 将数据放到gpu上进行计算

```
data, label = data.cuda(), label.cuda()
```

- 每批数据训练前, 都将优化器梯度置为零

```
optimizer.zero_grad()
```

- 将读取的数据送到模型中进行训练

```
output = model(data)
```

- 根据预先定义好的损失函数计算损失

```
loss = criterion(output, label)
```

- 反向传播

```
loss.backward()
```

- 优化器更新模型参数

```
optimizer.step()
```

训练流程

```
for epoch in range(max_epochs):  
    train(epoch) #训练函数  
    val(epoch) #验证/测试函数
```

- 训练一个epoch的函数

```
def train(epoch):  
    model.train()  
    train_loss = 0  
    for data, label in train_loader:  
        data, label = data.cuda(), label.cuda()  
        optimizer.zero_grad()  
        output = model(data)  
        loss = criterion(output, label)  
        loss.backward()  
        optimizer.step()  
        train_loss += loss.item()*data.size(0)  
    train_loss = train_loss/len(train_loader.dataset)  
    print('Epoch: {} \tTraining Loss: {:.6f}'.format(epoch, train_loss))
```

验证/测试流程

- 验证/测试的流程与训练过程基本一致，不同点在于：
 - 需要预先设置`torch.no_grad`，以及将`model`调至`eval`模式
 - 不需要将优化器的梯度置零
 - 不需要将`loss`反向回传到网络
 - 不需要更新`optimizer`

```
def val(epoch):  
    model.eval()  
    val_loss = 0  
    with torch.no_grad():  
        for data, label in val_loader:  
            data, label = data.cuda(), label.cuda()  
            output = model(data)  
            preds = torch.argmax(output, 1)  
            loss = criterion(output, label)  
            val_loss += loss.item()*data.size(0)  
            running_accu += torch.sum(preds == label.data)  
    val_loss = val_loss/len(val_loader.dataset)  
    print('Epoch: {} \tTraining Loss: {:.6f}'.format(epoch, val_loss))
```

验证/测试流程

- 对于图像分类任务，我们还可以使用sklearn.metrics中的classification_report函数来计算模型的准确率、召回率、F1值等指标

```
from sklearn.metrics import classification_report  
.....
```

将下方代码的labels和preds替换为模型预测出来的所有label和preds,
target_names替换为类别名称,
既可得到模型的分类报告

```
.....  
print(classification_report(labels.cpu(), preds.cpu(), target_names=class_names))
```

8. 可视化



使用print函数打印模型基础信息

```
import torchvision.models as models
model = models.resnet18()
print(model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  ... ..
```

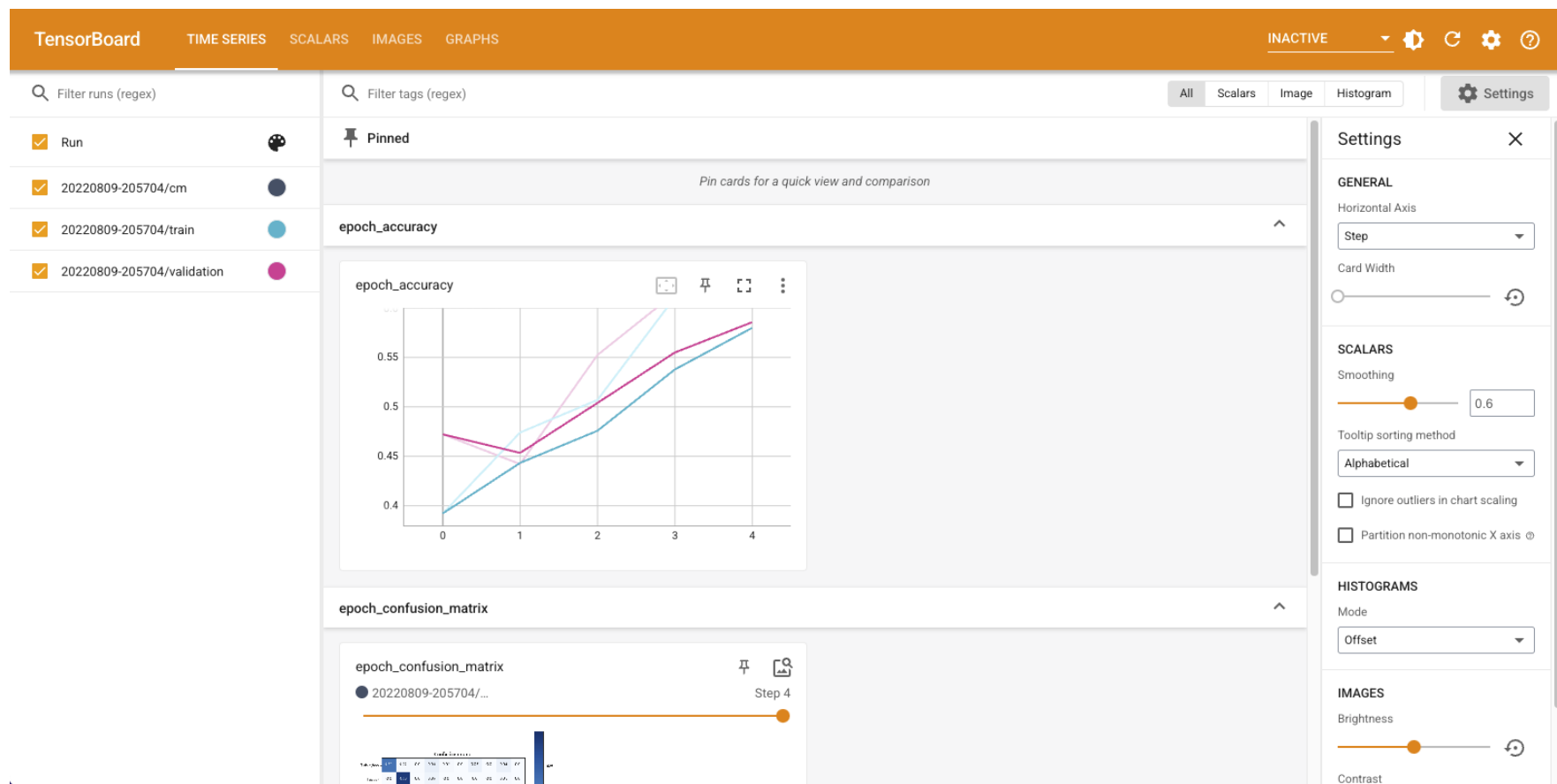
使用torchinfo输出模型结构

```
import torchvision.models as models
from torchinfo import summary
resnet18 = models.resnet18() # 实例化模型
summary(resnet18, (1, 3, 224, 224)) # 1: batch_size 3: 图片的通道数 224: 图片的高宽
```

Layer (type:depth-idx)	Output Shape	Param #
ResNet	--	--
├Conv2d: 1-1	[1, 64, 112, 112]	9,408
├BatchNorm2d: 1-2	[1, 64, 112, 112]	128
├ReLU: 1-3	[1, 64, 112, 112]	--
├MaxPool2d: 1-4	[1, 64, 56, 56]	--
├Sequential: 1-5	[1, 64, 56, 56]	--
│├BasicBlock: 2-1	[1, 64, 56, 56]	--
││├Conv2d: 3-1	[1, 64, 56, 56]	36,864
││├BatchNorm2d: 3-2	[1, 64, 56, 56]	128
││├ReLU: 3-3	[1, 64, 56, 56]	--
││├Conv2d: 3-4	[1, 64, 56, 56]	36,864
││├BatchNorm2d: 3-5	[1, 64, 56, 56]	128
││├ReLU: 3-6	[1, 64, 56, 56]	--
│├BasicBlock: 2-2	[1, 64, 56, 56]	--
││├Conv2d: 3-7	[1, 64, 56, 56]	36,864

使用Tensorboard可视化训练过程

- Tensorboard是一款可视化工具，可以可视化输入数据，模型结构，参数变化，损失函数等等



Tensorboard使用

- 安装

```
pip install tensorboardX
```

- 配置：需要先指定一个文件夹供TensorBoard保存记录下来的数据，上面的操作实例化SummaryWriter为变量writer，并指定writer的输出目录为当前目录下的"runs"目录

```
from tensorboardX import SummaryWriter  
  
writer = SummaryWriter('./runs')
```

- 命令行中的启动命令

```
tensorboard --logdir=/path/to/logs/ --port=xxxx
```

Tensorboard可视化模型

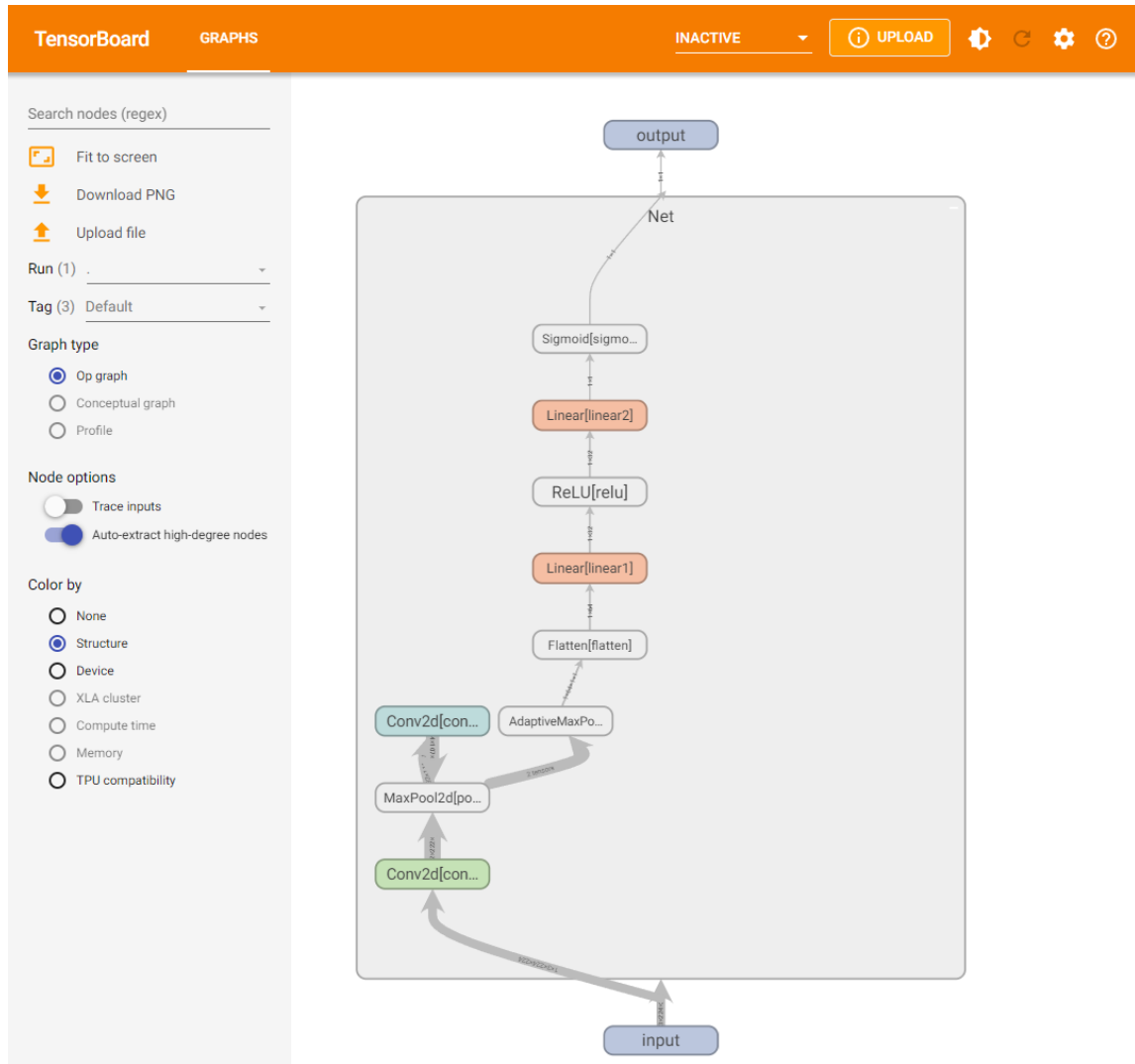
```
class Net(nn.Module):  
    def __init__(self):  
        super(Net, self).__init__()  
        self.conv1 = nn.Conv2d(in_channels=3,out_channels=32,kernel_size = 3)  
        self.pool = nn.MaxPool2d(kernel_size = 2,stride = 2)  
        self.conv2 = nn.Conv2d(in_channels=32,out_channels=64,kernel_size = 5)  
        self.adaptive_pool = nn.AdaptiveMaxPool2d((1,1))  
        self.flatten = nn.Flatten()  
        self.linear1 = nn.Linear(64,32)  
        self.relu = nn.ReLU()  
        self.linear2 = nn.Linear(32,1)  
        self.sigmoid = nn.Sigmoid()
```

```
writer.add_graph(model, input_to_model = torch.rand(1, 3, 224, 224))  
writer.close()
```

- 在Python代码中初始化好writer，并写入数据，运行完后，在命令行中用以下命令启动，可看到模型结构。

```
tensorboard --logdir=/path/to/logs/ --port=xxxx
```

Tensorboard可视化模型



Tensorboard可视化图像

```
train_data = datasets.CIFAR10(".", train=True, download=True, transform=transform_train)
test_data = datasets.CIFAR10(".", train=False, download=True, transform=transform_test)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64)
images, labels = next(iter(train_loader))
# 仅查看一张图片
writer = SummaryWriter('./pytorch_tb')
writer.add_image('images[0]', images[0])
writer.close()
# 将多张图片拼接成一张图片, 中间用黑色网格分割
writer = SummaryWriter('./pytorch_tb')
img_grid = torchvision.utils.make_grid(images)
writer.add_image('image_grid', img_grid)
writer.close()
# 将多张图片直接写入
writer = SummaryWriter('./pytorch_tb')
writer.add_images("images", images, global_step = 0)
writer.close()
```

```
tensorboard --logdir=/path/to/logs/ --port=xxxx
```


Tensorboard可视化图像

images

images

tag: images

step 0

Fri Feb 11 2022 15:25:03 GMT+0800 (中国标准时间)



Tensorboard可视化

```
writer = SummaryWriter('./pytorch_tb')
for i in range(500):
    x = i
    y = x**2
    writer.add_scalar("x", x, i) #日志中记录x在第step i 的值
    writer.add_scalar("y", y, i) #日志中记录y在第step i 的值
writer.close()
```

WALL

Runs

Write a regex to filter runs

☒ ☐ .

TOGGLE ALL RUNS

/data1/ljq/jupyter_code/pytorch_tb

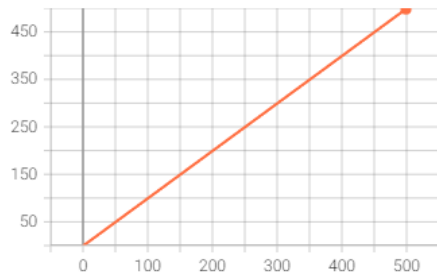
- 在Python代码中初始化好writer，并写入数据，运行完后，在命令行中用以下命令启动，可看到数据变化

```
tensorboard --logdir=/path/to/logs/ --port=xxxx
```

Filter tags (regular expressions supported)

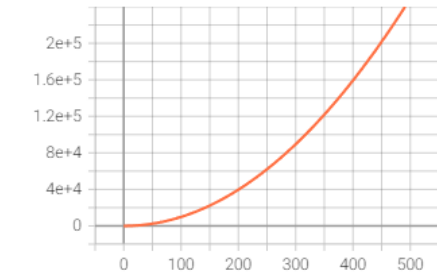
x

x
tag: x



y

y
tag: y



THANK YOU!

Jifeng Dai
Department of Electronic Engineering
daijifeng@tsinghua.edu.cn

